

From Majority Vote to Bias-Aware Decoding: A Layered Repetition-Code Study with Stim Simulations and Constrained IBM Quantum Validation

Abstract

This paper presents a layered study of repetition-code quantum memories that connects local simulation rigor to real-hardware execution. Layer 1 reproduces a Qiskit Aer baseline using majority-vote decoding under bit-flip, phase-flip, and depolarizing noise with explicit idle periods. Layer 2 upgrades the workflow to Stim circuit-level repetition-memory simulations with detector-model decoding via PyMatching minimum-weight perfect matching (MWPM), and extends analysis to biased noise, heterogeneous per-qubit rates, temporal drift, and decoder benchmarking. Layer 3 executes constrained IBM Quantum validation on backend `ibm_fez` with an $n=3$ protocol (3 circuits, 1000 shots/circuit), repeated across three hardware jobs for short-horizon repeatability. Local results show expected logical-error suppression with distance under detector-aware decoding, while hardware runs return logical-success probabilities in $[0.999, 1.000]$ with maximum run-to-run per-circuit variation 0.001. The key outcome is an end-to-end reproducible pipeline: baseline reproduction, modern detector-graph decoding, structured robustness probes, and verified cloud execution.

1. Introduction

Quantum error correction (QEC) progress depends on both algorithmic quality (decoder effectiveness) and systems quality (repeatable execution under realistic runtime constraints). The repetition code is a limited but practical testbed for this purpose: it isolates syndrome extraction and decoding behavior while keeping circuit structure and interpretation simple.

This work was designed around three requirements:

1. Reproduce a transparent baseline that can be sanity-checked analytically.
2. Upgrade to circuit-level simulation and detector-model decoding to test realistic scaling behavior.
3. Demonstrate that the same workflow can be executed on cloud hardware and yield stable artifacts.

The result is a layered methodology rather than a single benchmark number. That methodology is the primary contribution.

2. Scope and Contributions

This project contributes:

1. A corrected and reproducible Layer 1 Qiskit baseline with preserved idle-gate behavior for noise application.
2. A Layer 2 Stim plus PyMatching study across distance scaling, biased noise, heterogeneous noise, temporal drift, and decoder comparisons.
3. A decoder benchmark framing that distinguishes deliberately weak majority decoding from detector-aware MWPM and an unweighted MWPM control.
4. A completed Layer 3 constrained IBM execution path with metadata capture, figure generation, and three-run repeatability summaries.

3. Technical Background

For an odd-distance repetition code, logical readout in the protected basis can be decoded by majority vote on data outcomes, but that rule ignores detector-graph structure and measurement-history correlations. Detector-based decoding maps syndrome data to a graph optimization problem solved by MWPM.

We report logical success probability P_{succ} and logical error rate

$$P_{\mathrm{err}} = 1 - P_{\mathrm{succ}}.$$

In Layer 2 and Layer 3 summaries, comparisons are made at fixed shot budgets and aligned circuit families.

4. Methods

4.1 Layer 1: Qiskit Aer baseline

Configuration:

1. Code sizes: $n \in \{1, 3, 5\}$.
2. Noise channels: bit-flip, phase-flip, depolarizing.
3. Four idle steps, each with per-step physical error probability p .
4. Decoder: majority vote.

Implementation note: transpilation was constrained to preserve idle/noise-relevant structure (basis-gate alignment with optimization suppression), preventing accidental elimination of identity

segments needed by the noise model.

4.2 Layer 2: Stim and PyMatching upgrade

Configuration:

1. Distances: $d \in \{3, 5, 7, 9, 11\}$.
2. Rounds: rounds $= d$ by default.
3. Circuit family: Stim repetition-memory circuits.
4. Decoder: PyMatching built from Stim detector error models.

Additional robustness axes:

1. Biased Pauli noise via p_Z/p_X sweeps.
2. Heterogeneous per-qubit noise via lognormal spreads around $p_{\{\text{mean}\}}$.
3. Temporal drift snapshots with synthetic quality perturbations.
4. Decoder comparison among majority vote, unweighted MWPM, and detector-model MWPM (plus optional toy neural baseline).

4.3 Layer 3: constrained IBM hardware validation

Protocol:

1. Backend: `ibm_fez`.
2. Three circuits per job: `z_idle_2`, `z_idle_4`, `x_idle_4`.
3. Shots: 1000 per circuit.
4. Three completed jobs: `danhfqh8gn2s739mlcp0`, `danhhqg2fm4c73f47lfg`, `danhhhlr85ps73fe2ffg`.

Artifacts:

1. Hardware result CSV.
2. Job metadata JSON.
3. Hardware validation figure.
4. Repeatability summary CSVs (pairwise and three-run aggregate).

5. Results

5.1 Baseline reproduction (Layer 1)

At $p=0.1\%$ (bit-flip slice), logical success increases with code size:

1. $n=1$: 0.7125
2. $n=3$: 0.79375
3. $n=5$: 0.8505

Across theory-supported bit-flip and phase-flip slices, mean absolute simulation-to-analytic gap is approximately 0.0057.

5.2 Distance scaling under detector-model decoding (Layer 2)

At $p=0.01$, logical error decreases monotonically with distance:

1. $d=3$: 0.00681
2. $d=5$: 0.00142
3. $d=7$: 0.00027
4. $d=9$: 0.00006
5. $d=11$: 0.00002

This is the central Layer 2 behavior: detector-aware MWPM restores expected low-noise suppression with increasing distance.

5.3 Biased-noise behavior

At distance 5 and total physical error probability 0.01, logical error drops as Z bias increases:

1. $b_Z=0.1$: 0.00141
2. $b_Z=1$: 0.00048
3. $b_Z=3$: 0.00019

Strongly Z -biased cases reach the 10^{-5} to 10^{-4} range in this approximation.

5.4 Heterogeneous-noise behavior

As spread increases, performance generally degrades relative to uniform assumptions. High-spread settings at larger physical error rates are best interpreted as stress tests for calibration non-uniformity, not clean scaling points.

5.5 Temporal-drift snapshots

At distance 11 and $p_{\mathrm{mean}}=0.05$:

1. Nominal: 0.10207

2. Single-bad-qubit perturbation: 0.16015
3. Global drift-up: 0.25173
4. Partial recovery snapshot: 0.12284

These snapshots demonstrate practical sensitivity to non-stationary calibration conditions.

5.6 Decoder comparison

At distance 11 and $p=0.02$:

1. Majority vote logical error: 0.2244
2. Detector-model MWPM logical error: 0.0006

This gap exceeds two orders of magnitude, emphasizing that detector-graph information is not a marginal improvement but a dominant factor in this regime.

5.7 Hardware validation and repeatability (Layer 3)

Across three constrained hardware runs:

1. All measured logical-success probabilities lie in $[0.999, 1.000]$.
2. Per-circuit max range across runs is 0.001.
3. Aggregate means: 0.9997 (z_idle_2), 0.9997 (z_idle_4), 1.0000 (x_idle_4).

Interpretation: at this shot budget and scope, observed variation is consistent with finite-shot fluctuation, and the end-to-end cloud pipeline is operational.

5.8 Additional experimental extension: Stim decoder comparison and hardware trials

To strengthen the study beyond the basic majority-vote baseline, we repeated a portion of the repetition-code experiments using Stim-generated circuits and decoder-aware matching. This additional experimentation was intended to answer a practical question: if we use the same repetition-code memory with a larger and more realistic circuit model, does the decoder matter as much as the code size itself? The answer is yes. In our Stim-based simulations, a detector-model minimum-weight perfect matching decoder produced substantially better logical performance than a simple majority-vote decoder, even when the underlying logical circuit family was the same.

At distance 11 and physical error rate $p = 0.02$, the majority-vote baseline produced a logical error rate of 0.2244, while the detector-model MWPM decoder reduced the logical error rate to 0.0006.

This is a difference of more than two orders of magnitude and shows that decoder quality can dominate basic post-processing choices. In other words, the repetition code behaves differently depending on whether the measurement record is interpreted by a minimal majority rule or by a graph-based decoding algorithm that incorporates syndrome structure. This result is especially

important because it demonstrates that the main performance gains are not only from encoding more qubits, but also from interpreting the resulting syndrome information more intelligently.

We also tested an unweighted MWPM comparison as an intermediate control. This is useful because it isolates the effect of decoder information from the effect of code geometry. The detector-model weighted decoder was clearly superior to the unweighted variant, confirming that the syndrome graph itself carries useful error information and that a decoder which uses that structure can recover more logical information. This reinforces the central idea that encoding and decoding must be considered together when evaluating quantum memory performance.

The second additional experiment involved direct IBM Quantum hardware execution. While the original project focused primarily on local simulation, we also ran a constrained hardware validation to check whether the same circuit family could be executed in a realistic cloud environment. In this trial, the backend was ``ibm_fez``, and each job contained three minimal repetition-memory circuits: ``z_idle_2``, ``z_idle_4``, and ``x_idle_4``. Each circuit was run with 1000 shots, and the full protocol was repeated across three jobs to evaluate short-horizon stability.

Across the three hardware trials, the logical success probability was nearly ideal for all circuits. The measured values were 0.999 for ``z_idle_2``, 1.000 for ``z_idle_4``, and 1.000 for ``x_idle_4`` in the latest run, and the maximum per-circuit run-to-run variation across all three jobs was only 0.001. This indicates that the hardware behavior was highly consistent for this minimal validation setting and that the cloud execution path, metadata logging, and result extraction were all functioning correctly. Even with heavy constraints on circuit size and shot count, the hardware results were stable enough to support the broader conclusion that the workflow is operational beyond pure simulation.

Taken together, these additional experiments improve the study in two important ways. First, they show that decoder design is a major determinant of logical performance in realistic repetition-memory circuits. Second, they show that the same pipeline can be run on real IBM hardware under constrained conditions with stable outputs. The simulations and hardware runs therefore complement one another: the local results establish a clean scaling analysis, while the hardware trials demonstrate that the methodology can be carried into a real device setting without losing reproducibility or scientific clarity.

6. Discussion

The layered design produces clearer attribution than a single monolithic experiment:

1. Layer 1 validates baseline mechanics and analytic consistency.
2. Layer 2 demonstrates where decoding sophistication materially changes outcomes.
3. Layer 3 verifies practical execution viability under real runtime constraints.

A key methodological lesson is that benchmark fairness requires shared syndrome samples across decoder methods whenever possible. Another is that integration details (transpilation choices, runtime API compatibility, and metadata capture) materially affect scientific reliability.

7. Limitations

1. Repetition code protects a restricted logical subspace and is not a full arbitrary-state code.
2. Hardware validation is intentionally constrained (single backend family, three circuits, 1000 shots each).
3. The toy neural decoder is exploratory and not a production learned-decoder baseline.
4. No claim is made here about thresholds or cross-platform universality.

8. Reproducibility

The repository provides script-driven execution for baseline, Stim upgrades, decoder comparisons, and IBM validation, with outputs committed as structured CSV/JSON/figure artifacts. Tests currently pass (8/8), supporting code-level reproducibility of the implemented pipeline.

9. Conclusion

This work delivers a complete, practical QEC mini-pipeline from classical baseline checks through detector-aware simulation to verified cloud execution. The strongest empirical message is the decoder effect size in Layer 2; the strongest systems message is that constrained Layer 3 execution is now real, repeatable, and artifact-complete.

10. Recommended Next Experiments

1. Multi-backend Layer 3 replication to estimate backend-to-backend variance.
2. Higher-shot repeats to tighten confidence intervals on near-unity success probabilities.
3. Expanded circuit families beyond $n=3$ constrained validation.
4. Stronger learned-decoder baseline trained and evaluated on detector-consistent datasets.

References

- [1] C. Gidney, Stim: a fast stabilizer circuit simulator.
- [2] O. Higgott, PyMatching: sparse blossom decoding for quantum error correction.
- [3] A. W. Cross et al., Open quantum assembly language and practical QEC benchmarking workflows.

[4] IBM Quantum documentation, Qiskit Runtime primitives and backend execution guidance.